Resultados do AprendFed

LeanFed-c contra FedAvg com bateria limitada: o que os experimentos mostram

Até 25 de outubro de 2026 — CIFAR-10 e FEMNIST com 8 seeds — branch *leanfed-baseline* (commit 86ab3a9); HeteroFL na branch *heterofl* (commit f13bb77)

Resumo. Com bateria simulada e custo de comunicação realista, o LeanFed original **não melhora** sobre o FedAvg (+0,3 ponto com 8 seeds). A versão corrigida (**LeanFed-c**), que reserva a comunicação e o custo fixo antes de escolher quantos dados treinar, mantém praticamente todos os clientes vivos até o fim. Com 8 seeds no CIFAR-10, isso vale **+9,1 pontos** de acurácia com Dirichlet α = 0,05 (positivo nas 8 seeds) e **+4,3** com α = 0,1 (6 de 8; IC 95% de −0,1 a +8,8). Quando os dados de cada classe estão espalhados entre os clientes, o ganho some: +0,6 com α = 0,3 e −0,4 com α = 1,0. Pular as rodadas em que o cliente treinaria menos da metade dos dados (**fração mínima 0,5**) melhora o LeanFed-c **nos quatro α**, de +7,3 pontos (α = 0,05) a +0,5 (α = 1,0): sobre o FedAvg, ganha +16,3, +6,8 e +1,6 de α = 0,05 a α = 0,3 e empata com α = 1,0, onde o LeanFed-c sozinho perdia. No FEMNIST, com a acurácia medida depois de recalibrar a BatchNorm (seção 1), ela também é a melhor: +4,0 pontos sobre o FedAvg com Dirichlet α = 0,1 (8 de 8 seeds) e +0,2 na partição por escritor, onde os três métodos ficam a menos de meio ponto. Com ela, os clientes de bateria pequena pulam o começo do treino e chegam ao fim com energia para treinar todos os dados; quanto maior a fração mínima, maior o ganho (com 1,0, +8,5 sobre o FedAvg em α = 0,1). Esse ganho extra se espalha por todas as classes, e não só pelas desses clientes, e sobe sobretudo a pior classe. Ele vem de treinar mais exemplos no total: cada rodada pulada economiza a energia de rede, que vira treino. Sem rádio, o total fica igual e o ganho cai para +1,2 ponto (fração 0,5, sem significância); com a margem de cauda, ninguém morre e o ganho fica (+2,9). Variando a potência do rádio, o ganho acompanha o custo da rede: +0,4 com 0,9 W, +2,5 com 1,8 W e +3,9 com 2,7 W (8 de 8 seeds). Com o rádio diferente entre clientes (média 1,8 W), ele fica (+1,6) quando o rádio mais caro está nos clientes de menos bateria e some (+0,5) quando está nos de mais. Entre cenários, o ganho acompanha a fração de classes concentradas nos clientes que o FedAvg perde por falta de bateria, e não a quantidade de dados que esses clientes levam; seed a seed, essa fração não prevê o ganho. Reduzir o modelo em vez dos dados (**HeteroFL**, submodelos de largura variável) **perde** no α = 0,1: 67,7% contra 74,7% do LeanFed-c e 78,0% da fração 0,5, abaixo dos dois nas 8 seeds. Ele treina mais exemplos (780 mil, contra 563 mil), mas só 348 mil passam pelo modelo completo, e a perda se espalha por todas as classes, sem ficar nas dos clientes de bateria pequena. Só com as larguras 1 e 3/4, ele sobe para 72,7% (+5,0, 8 de 8 seeds), mas continua abaixo do LeanFed-c (−2,0) e da fração 0,5 (−5,3).

Conteúdo

- 1. Cenário, políticas e métricas
- 2. O LeanFed-c ganha porque não perde clientes no fim
- 3. Ajustes do LeanFed-c: a fração mínima ajuda em todos os cenários
- 4. O ganho depende da concentração de classes

- 5. Reduzir o modelo (HeteroFL) perde para reduzir os dados
- 6. Conclusões, limitações e próximos passos
- Apêndice A. Runs e como reproduzir

A ordem dos experimentos e o motivo de cada decisão estão em *historico_projeto.pdf*, na mesma pasta.

1. Cenário, políticas e métricas

Cenário de referência. FedAvg com Flower, um container Docker por cliente. 15 clientes, baterias de 1500/1000/500 J (5 clientes de cada), rede de 25 Mbps, MobileNetV3-Small pré-treinada a 96 px, 20 rodadas, uma época local, todos os clientes sorteados em toda rodada. Dataset padrão: CIFAR-10 dividido por Dirichlet $\alpha = 0{,}1$.

Energia. O treino custa 8,88 mJ por amostra (perfil medido na GPU com NVML, acima do repouso). A comunicação é potência do rádio (1,8 W) × bits / vazão, com a vazão sorteada em U(1, 25) Mbps a cada rodada: em média 23,6 J por rodada, com cauda longa (quase 100 J com vazão perto de 1 Mbps). Quando a bateria acaba, o cliente morre no meio da rodada.

Política	Fração η dos dados treinada por rodada	Ideia
FedAvg	1	treina tudo até a bateria acabar
LeanFed	orçamento / A	LeanFed original: supõe que só o treino gasta energia
LeanFed-c	(orçamento − C − D) / A	LeanFed corrigido: reserva antes a rede (C) e o custo fixo (D)
HeteroFL	1, num submodelo de largura p	reduz o modelo em vez dos dados (seção 5)

O orçamento da rodada é a bateria atual dividida pelas participações que faltam; A é a energia de treinar todos os dados locais. Com $\eta = 0$, o cliente pula a rodada sem gastar nada.

Métricas.

- **Acurácia**: média da acurácia no teste nas **3 últimas rodadas**. A da última rodada sozinha oscila vários pontos conforme quais clientes ainda estão vivos. No FEMNIST e na comparação com o HeteroFL (seção 5), medida com a BatchNorm recalibrada (abaixo).
- **Ganho**: acurácia do LeanFed-c menos a do FedAvg **com a mesma seed** (mesma partição e mesma sequência de vazões), com intervalo de confiança de 95% pela t de Student sobre as diferenças pareadas.
- **Seeds**: valores são média ± desvio amostral sobre as seeds 42 a 49 (n = 8), em todos os cenários. Diferenças entre duas variantes do LeanFed-c também são pareadas por seed.
- **Cliente perdido**: não contribuiu em nenhuma das 3 últimas rodadas. **Classes concentradas**: fração das classes com mais de 80% dos seus exemplos de treino nos clientes perdidos pelo FedAvg.

Avaliação no FEMNIST: BatchNorm recalibrada

A FedAvg tira a média das estatísticas das camadas BatchNorm (média e variância das ativações) junto com a dos pesos. No FEMNIST não-IID, essa média não representa os dados, e a acurácia do modelo global no teste fica subestimada e às vezes desaba numa rodada, com todos os clientes treinando normalmente: na rodada 1, 9% contra 33% recalibrado (FedAvg com Dirichlet, média das seeds); das rodadas 5 a 20, 32 das 768 avaliações caem abaixo de 30%, a pior a 2,4%.

Por isso, nas runs do FEMNIST o servidor também avalia o modelo depois de refazer essas estatísticas com 2048 exemplos de treino dos clientes, sorteados pela seed (sem usar o teste). Os pesos não mudam, e o modelo enviado aos clientes também não: a recalibração só entra na medida. Com ela, a pior avaliação das rodadas 5 a 20 fica em 57,6%, e a média das 3 últimas rodadas sobe de 5 a 16 pontos, conforme o método. Todas as acurácias do FEMNIST neste documento são as recalibradas.

> **Sem recalibrar, o FEMNIST não compara métodos.** O desvio entre seeds chega a 18 pontos (contra 2 recalibrado), e a ordem muda: com Dirichlet, a fração mínima 0,5 ficaria em último (62,7% contra 69,5% do FedAvg); por escritor, em primeiro (78,8% contra 74,6%). No CIFAR-10, as quedas de acurácia são reais (vêm dos clientes que saem) e recalibrar não as recupera; ele usa a avaliação original, fora da seção 5.

2. O LeanFed-c ganha porque não perde clientes no fim

CIFAR-10, α = 0,1, seeds 42 a 49	Média das 3 últimas	Acurácia máxima	Rodadas *dead*	Sem bateria no fim
FedAvg	70,4 ± 6,0%	78,2 ± 1,4%	45,5	7,8
LeanFed	70,8 ± 6,5%	77,6 ± 1,6%	29,4	7,8
LeanFed-c	**74,8 ± 3,6%**	77,5 ± 2,5%	0,8	1,6

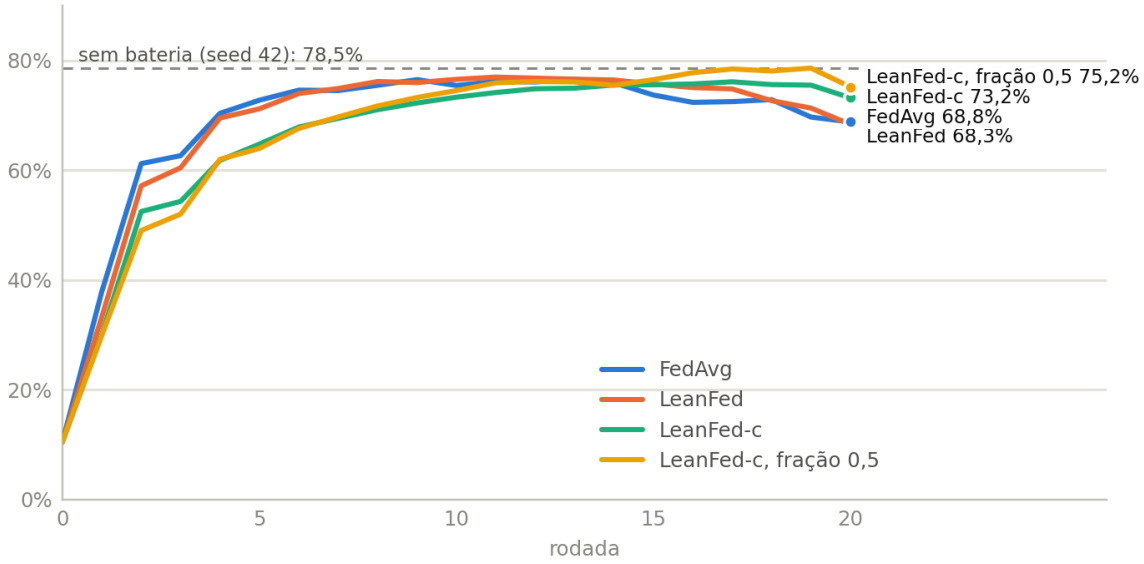


Figura 1. Acurácia no teste por rodada, média das seeds 42 a 49 (CIFAR-10, α = 0,1). FedAvg, LeanFed e LeanFed-c chegam a picos parecidos (76,6%, 77,0% e 76,1%). A diferença está no fim, quando os clientes de bateria pequena morrem no FedAvg e no LeanFed. O LeanFed-c com fração mínima 0,5 (seção 3) chega a um pico maior (78,6%). Todas as curvas caem na rodada 20.

- **O LeanFed-c ganha em 6 das 8 seeds**: +10,8, +2,0 e +13,0 pontos nas seeds 42 a 44; −1,8, +4,1, +1,7, +5,8 e −1,0 nas seeds 45 a 49. A média cai de +8,6 (n = 3) para **+4,3**, com IC 95% de −0,1 a +8,8 (t pareado, p = 0,055).
- **O pico não melhora**: 77,5% no LeanFed-c contra 78,2% no FedAvg. O ganho vem de não perder acurácia nas últimas rodadas, e não de aprender mais.
- **O LeanFed original não ajuda**: +0,3 ponto sobre o FedAvg (IC 95% de −0,6 a +1,3). Gasta o orçamento todo em treino, paga a rede por fora e termina com os mesmos 7,8 clientes sem bateria do FedAvg. Morre menos rodadas no meio do caminho (29 contra 45), mas isso não muda a acurácia. Aqui a comunicação é de um terço a metade do custo de uma rodada.
- **A energia total é a mesma** (11,3 kJ no FedAvg, 11,2 no LeanFed-c): o LeanFed-c distribui a bateria ao longo das 20 rodadas, com 285 participações contra 247 do FedAvg.

Por que sobram perdas. Na seed 43 (ganho de só 2 pontos), três clientes do LeanFed-c morrem nas rodadas 18 a 20, cada um por **uma** rodada de vazão perto de 1 Mbps (52 a 127 J de rede contra 23,6 J reservados), quando a bateria já não tinha folga. Dois deles tinham 94% dos pássaros e 92% dos cervos, que caem a 48% e 18% de acurácia no modelo final.

Quando o LeanFed-c perde. Nas seeds 45 e 49, o FedAvg quase não cai no fim: na 49, a última rodada é o seu máximo (78,0%); na 45, ele fica em 77,3% nas 3 últimas, embora perca 74% dos dados e 60% das classes fiquem concentradas nos clientes que saem. O LeanFed-c, que treina só parte dos dados por rodada, chega a picos menores (79,6% e 77,8% contra 81,0% e 78,0%) e, na seed 45, ainda perde 2 clientes no fim. Com a fração mínima 0,5 (seção 3), as duas seeds passam a ganhar (+2,5 e +0,1).

3. Ajustes do LeanFed-c: a fração mínima ajuda em todos os cenários

A **fração mínima** faz o cliente pular a rodada quando a fração η que cabe no orçamento fica abaixo do mínimo. A energia não gasta passa para as rodadas seguintes. Na última participação, o mínimo não vale. Com fração mínima 1,0, o cliente treina todos os dados ou pula.

Variante (α = 0,1, 8 seeds)	Média das 3 últimas	Diferença para o LeanFed-c [IC 95%], seeds melhores	Acurácia máxima	Particip.	Rodadas *dead*
LeanFed-c	74,8 ± 3,6%	—	77,5 ± 2,5%	285	0,8
fração mínima 0,3	76,6 ± 2,2%	**+1,9** [+0,6; +3,2], 7 de 8	79,4 ± 2,1%	270	0,4
fração mínima 0,5	77,3 ± 2,7%	**+2,5** [+1,0; +4,1], 7 de 8	80,1 ± 1,9%	260	0,4
fração mínima 0,7	77,6 ± 3,4%	**+2,9** [+1,9; +3,8], 8 de 8	80,0 ± 1,9%	251	0,4
fração mínima 1,0	**78,9 ± 3,1%**	**+4,1** [+1,9; +6,4], 8 de 8	80,0 ± 2,9%	240	0
margem de cauda 0,9	75,4 ± 3,6%	+0,6 [−0,8; +2,1], 4 de 8	77,3 ± 2,9%	278	0

- **A fração mínima é o único ajuste que melhora a acurácia**, e o ganho cresce com o mínimo: +1,9, +2,5, +2,9 e +4,1 pontos sobre o LeanFed-c com 0,3, 0,5, 0,7 e 1,0 (t pareado, p de 0,012 a < 0,001). Com 0,5, o efeito é o mesmo nas seeds 42 a 44 (+2,0) e nas 45 a 49 (+2,8). Com 3 seeds, esse ganho parecia ruído.
- **Treinar uma fração dos dados não ajuda.** A melhor variante é a 1,0, que treina tudo ou pula: 78,9%, nenhuma rodada *dead* e +8,5 pontos sobre o FedAvg (IC 95% de +5,3 a +11,7; 8 de 8). A diferença para a 0,5 (+1,6; IC 95% de −0,3 a +3,6; p = 0,09) não é significativa, mas não há nenhum sinal a favor de frações menores. O que ajuda é pular rodadas para acumular energia. A fração 0,5 segue como valor de referência, e a varredura entra como análise de sensibilidade.
- **Contra o FedAvg, a fração mínima 0,5 ganha nas 8 seeds**: +6,8 pontos (IC 95% de +2,6 a +11,1; p = 0,007), contra +4,3 do LeanFed-c sem ela. As seeds 45 e 49, em que o LeanFed-c perdia, ficam em +2,5 e +0,1.
- **Diferente do LeanFed-c, a fração mínima também sobe o pico**: 80,1% contra 78,2% do FedAvg com 0,5 (+1,8 ponto pareado, 7 de 8 seeds, p = 0,002); 80,0% com 0,7 e 1,0.
- **Margem de cauda** (guardar o p90 da comunicação até a última participação): zera as mortes, mas não muda a acurácia (+0,6 ponto, 4 de 8 seeds). Depois de uma rodada de rede ruim no fim, o cliente pula as últimas rodadas em vez de morrer, e as classes dele somem do modelo do mesmo jeito.
- **Reserva por percentil** em vez da média (seed 42): o p90 evita mortes mas corta 47 participações; o p50 volta a perder clientes. A média ficou como padrão.

Custo fixo por rodada. Com D = 5 J (D ≈ C/5, como no exemplo do documento de custo de energia), FedAvg e LeanFed perdem cerca de 2,5 pontos, piores nas 8 seeds (67,9 ±

6,3% e 68,4 ± 6,3%), e o LeanFed morre mais (29 → 42 rodadas *dead*). O LeanFed-c, que já reserva D, fica igual (74,2 ± 3,3%; −0,5 ponto, não significativo). A vantagem dele sobre o FedAvg sobe para **+6,3 pontos** (IC 95% de +0,8 a +11,8; 6 de 8 seeds). Com 3 seeds, parecia de 10.

O que a fração mínima muda na participação

Os logs dos clientes mostram quando os 5 clientes de 500 J treinam (α = 0,1, média das 8 seeds). Os de 1000 e 1500 J participam de quase todas as rodadas em todas as políticas.

Clientes de 500 J	Ativos nas rodadas 1 a 7	Ativos nas rodadas 18 a 20	Exemplos treinados por rodada, rodadas 18 a 20
FedAvg	98%	8%	0,1 mil
LeanFed-c	91%	78%	3,8 mil
fração mínima 0,5	31%	85%	8,1 mil
fração mínima 0,7	23%	85%	9,4 mil
fração mínima 1,0	17%	94%	11,5 mil

- O FedAvg gasta a bateria dos clientes de 500 J no começo e chega ao fim sem eles. O LeanFed-c os mantém vivos, mas cada um treina pouco por rodada (no fim, 3,8 mil exemplos somados entre os 5).
- Com a fração mínima, esses clientes pulam o começo, acumulam energia e treinam quase todos os dados nas últimas rodadas. Quanto maior o mínimo, mais a participação se concentra no fim, na mesma ordem do ganho da tabela acima.
- É o oposto de fazer os clientes frágeis contribuírem mais cedo: com a fração mínima, eles contribuem **tarde**, e com mais dados.

Acurácia por classe: o ganho não fica nas classes desses clientes

Para ver se o ganho vem das classes dos clientes de 500 J, o modelo final de cada run foi avaliado por classe no teste do CIFAR-10 (α = 0,1, 8 seeds). "Classes dos clientes de 500 J" são as que têm mais da metade dos exemplos de treino nesses clientes: de 2 a 4 por seed, 29% do total. Só existe o modelo da rodada 20, a da queda em todas as curvas, por isso a acurácia global é um pouco menor que a média das 3 últimas rodadas.

α = 0,1, rodada 20	Global	Classes dos clientes de 500 J	Outras classes	Pior classe
FedAvg	68,8%	68,0%	68,7%	21,2%
LeanFed-c	73,2%	75,1%	72,4%	32,8%
fração mínima 0,5	75,2%	77,1%	74,2%	39,6%
fração mínima 0,7	76,0%	81,3%	73,6%	40,0%
fração mínima 1,0	78,6%	80,1%	77,3%	46,2%

- **O ganho sobre o LeanFed-c se espalha.** A fração mínima 0,5 sobe +2,0 pontos nas classes dos clientes de 500 J e +1,9 nas outras. Essas classes levam 37% do ganho,

quase a parte delas no total (29%), e a correlação entre o ganho de cada classe e a parte dela nesses clientes é fraca (Spearman +0,07 em média por seed; +0,15 com a fração 1,0), sem significância. Com α = 0,05 há alguma concentração (62% do ganho em 31% das classes), com intervalo largo.

- **A pior classe é a que mais sobe.** Com a fração 1,0, +13,5 pontos sobre o LeanFed-c (IC 95% de +5,3 a +21,6; 7 de 8 seeds) e +25,0 sobre o FedAvg (+8,5 a +41,5), contra +5,4 e +9,8 na acurácia global. Com a fração 0,5, +6,8 sobre o LeanFed-c, sem significância.
- **Então o ganho não vem das classes desses clientes.** A participação deles muda como a tabela anterior mostra, mas o ganho se espalha. A subseção seguinte mostra de onde ele vem.

De onde vem o ganho: mais exemplos treinados no total

Havia duas hipóteses. (1) As atualizações treinadas com poucos dados do LeanFed-c atrapalham o modelo global. Ela é fraca desde o início: a agregação já pesa cada cliente pelos exemplos que ele treinou na rodada. (2) A fração mínima faz entrar mais exemplos no treino. Os logs dos clientes mostram que entram: no α = 0,1, de 505 mil exemplos treinados em 20 rodadas no LeanFed-c para 563 mil com a fração 0,5 e 583 mil com a 1,0, na mesma ordem do ganho.

Isso acontece porque cada participação paga a rede (cerca de 23,5 J para baixar e enviar o modelo), seja qual for a fração dos dados treinada. Quando o cliente pula uma rodada, ele não gasta essa energia e pode usá-la para treinar. Com a fração 1,0, a energia de rede cai de 6,7 para 5,7 kJ e a de treino sobe de 4,5 para 5,2 kJ. Dois experimentos isolam esse efeito (α = 0,1, 8 seeds):

- **Sem rádio** (*--radio-w 0*): a rede não custa nada, e pular uma rodada deixa de economizar energia. As baterias caem para 1025, 550 e 85 J, para a energia de treino ficar parecida com a do caso com rádio.
- **Com margem de cauda 0,9** nas três políticas: ninguém fica sem bateria, e a diferença na morte de clientes entre o LeanFed-c (1,6 clientes) e a fração mínima sai da comparação.

α = 0,1	Variante	Exemplos em 20 rodadas	Por rodada, 18 a 20	Sem bateria no fim	Média das 3 últimas	Diferença para o LeanFed-c [IC 95%], seeds melhores
Com rádio	LeanFed-c	505 mil	25,4 mil	1,6	74,8 ± 3,6%	—
	fração 0,5	563 mil	31,2 mil	1,2	77,3 ± 2,7%	**+2,5** [+1,0; +4,1], 7 de 8
	fração 1,0	583 mil	38,5 mil	0,1	78,9 ± 3,1%	**+4,1** [+1,9; +6,4], 8 de 8
Sem rádio	LeanFed-c	529 mil	26,4 mil	0,0	76,0 ± 2,2%	—
	fração 0,5	529 mil	31,1 mil	0,0	77,2 ± 2,9%	**+1,2** [−0,1; +2,5], 7 de 8
	fração 1,0	516 mil	36,1 mil	0,0	76,6 ± 3,1%	**+0,6** [−1,5; +2,7], 4 de 8

α = 0,1	Variante	Exemplos em 20 rodadas	Por rodada, 18 a 20	Sem bateria no fim	Média das 3 últimas	Diferença para o LeanFed-c [IC 95%], seeds melhores
Margem de cauda	LeanFed-c	492 mil	27,7 mil	0,2	75,4 ± 3,6%	—
	fração 0,5	548 mil	33,5 mil	0,0	78,3 ± 2,5%	**+2,9** [+1,5; +4,4], 7 de 8
	fração 1,0	559 mil	38,4 mil	0,0	78,7 ± 3,8%	**+3,3** [+1,2; +5,5], 7 de 8

- **Sem rádio, o total de exemplos fica igual e o ganho quase some**: os três treinam cerca de 520 mil exemplos, e a fração mínima fica em +1,2 (0,5; p = 0,08) e +0,6 ponto (1,0; p = 0,5), contra +2,5 e +4,1 com rádio. A participação continua concentrada no fim (36 mil exemplos por rodada nas últimas rodadas com a fração 1,0, contra 26 mil no LeanFed-c), e a fração 1,0 é a que mais perde. Concentrar os dados no fim, sozinho, explica pouco.
- **Com margem de cauda, o ganho fica**: +2,9 (0,5; p = 0,002) e +3,3 pontos (1,0; p = 0,008) sobre o LeanFed-c com a mesma margem, sem ninguém sem bateria nas variantes com fração mínima. A diferença de exemplos também fica (492 mil contra 548 e 559 mil). A morte de clientes no fim do LeanFed-c não explica o ganho.
- **O ganho vem de treinar mais exemplos no total**, com a energia que a fração mínima deixa de gastar com a rede. Por isso ele não fica nas classes dos clientes de 500 J: o que muda é a quantidade de dados que entra no treino inteiro. Com a margem de cauda, a fração 0,5 chega a 78,3% (+1,0 ponto sobre ela sem margem, sem significância); a 1,0 fica igual (78,7%).
- Seed a seed, a relação é mais fraca. Com a fração 1,0, a seed que ganha mais exemplos sobre o LeanFed-c tende a ganhar mais acurácia (Spearman +0,71, p = 0,05); com a 0,5, não (+0,12). Com 8 seeds, uma diferença de ~60 mil exemplos (cerca de 10%) é pequena diante da variação entre seeds.

Se a causa é a rede, o ganho deve crescer com o custo dela. Para testar, a potência do rádio variou de 1,8 W para 0,9 e 2,7 W, com as mesmas baterias (1500, 1000 e 500 J), no LeanFed-c e na fração 0,5 (α = 0,1, 8 seeds).

Rádio	LeanFed-c	Fração 0,5	Exemplos em 20 rodadas	Rede (kJ)	Sem bateria no fim	Fração 0,5 − LeanFed-c [IC 95%], seeds melhores
0,9 W	77,8 ± 2,0%	78,2 ± 1,9%	644 → 656 mil	3,50 → 3,39	1,0 → 0,9	**+0,4** [−0,4; +1,2], 7 de 8
1,8 W	74,8 ± 3,6%	77,3 ± 2,7%	505 → 563 mil	6,71 → 6,15	1,6 → 1,2	**+2,5** [+1,0; +4,1], 7 de 8
2,7 W	71,4 ± 3,2%	75,3 ± 2,8%	403 → 472 mil	8,91 → 8,18	2,6 → 2,0	**+3,9** [+1,9; +5,9], 8 de 8

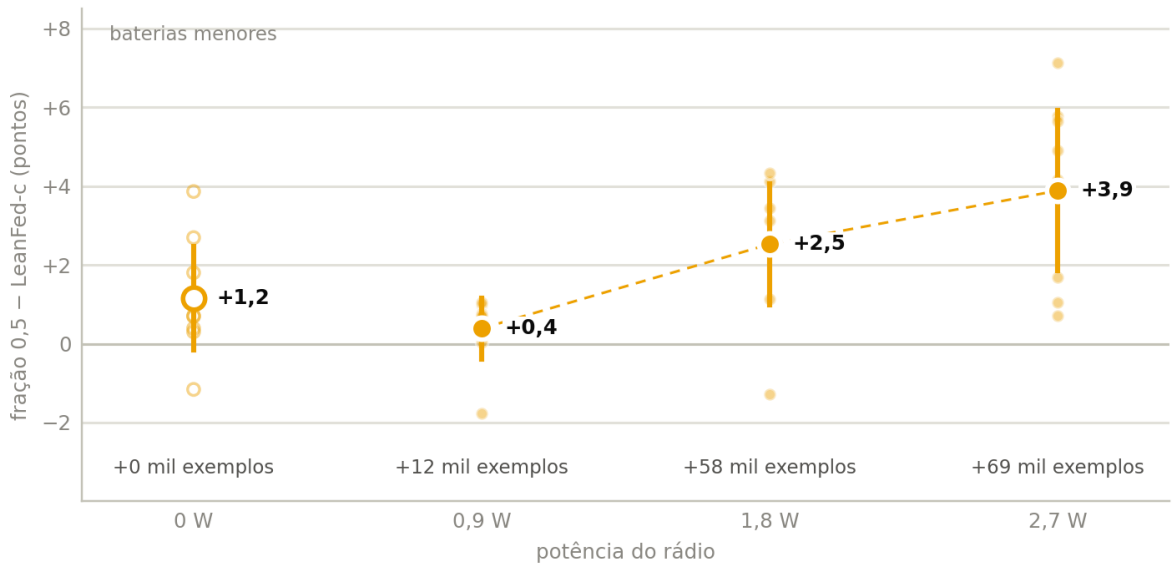


Figura 2. Ganho da fração mínima 0,5 sobre o LeanFed-c contra a potência do rádio (CIFAR-10, α = 0,1). Cada ponto pequeno é uma seed; o grande, a média, com o IC 95%. Embaixo, quantos exemplos a mais a fração 0,5 treina em 20 rodadas. O 0 W (vazado) usou baterias de 1025, 550 e 85 J e não entra na linha tracejada.

- **O ganho cresce com a potência**: +0,4 ponto com 0,9 W (sem significância, p = 0,27), +2,5 com 1,8 W e +3,9 com 2,7 W (8 de 8 seeds, p = 0,003). Junto cresce a folga de exemplos: +12, +58 e +69 mil.
- **Com rede barata, pular rodadas quase não economiza**: com 0,9 W, a rede custa 3,5 kJ no LeanFed-c e a fração 0,5 economiza só 0,1 kJ dela (0,6 kJ com 1,8 W; 0,7 kJ com 2,7 W). O ganho fica perto do caso sem rádio (+1,2).
- **Com rede cara, os dois pioram, mas o LeanFed-c piora mais**: de 1,8 para 2,7 W, ele cai de 74,8% para 71,4% e a fração 0,5 de 77,3% para 75,3%. Os clientes sem bateria no fim sobem para 2,6 e 2,0, mas, como a margem de cauda mostrou, as mortes explicam pouco do ganho.
- É a previsão da causa (A): o ganho da fração mínima mede quanto da bateria a rede consome, e some quando a rede é barata.

E se o custo da rede for desigual? Com a mesma média de 1,8 W, o rádio passou a variar entre os clientes, nas duas atribuições extremas: no pior caso, os clientes de 500 J têm o rádio de 2,7 W e os de 1500 J, o de 0,9 W; no favorável, o contrário. Os de 1000 J ficam com 1,8 W (α = 0,1, 8 seeds).

Rádio (1500 / 1000 / 500 J)	LeanFed-c	Fração 0,5	Exemplos em 20 rodadas	Exemplos dos clientes de 500 J	Sem bateria no fim	Fração 0,5 − LeanFed-c [IC 95%], seeds melhores
1,8 / 1,8 / 1,8 W (igual)	74,8 ± 3,6%	77,3 ± 2,7%	505 → 563 mil	38 → 91 mil	1,6 → 1,2	**+2,5** [+1,0; +4,1], 7 de 8
0,9 / 1,8 / 2,7 W (pior caso)	75,5 ± 2,8%	77,1 ± 2,6%	517 → 561 mil	34 → 73 mil	1,8 → 1,2	**+1,6** [+0,4; +2,8], 7 de 8
2,7 / 1,8 / 0,9 W (favorável)	76,4 ± 2,5%	76,9 ± 2,2%	558 → 577 mil	119 → 131 mil	1,4 → 1,2	**+0,5** [−1,1; +2,1], 3 de 8

- **A fração 0,5 não perde em nenhum caso, mas só ganha com clareza no pior caso**: +1,6 ponto (7 de 8 seeds, p = 0,02), contra +2,5 com o rádio igual. No favorável, +0,5, sem significância (3 de 8, p = 0,50).
- **A folga vem dos clientes de 500 J**: no LeanFed-c, com rádio de 2,7 W, eles treinam só 34 mil exemplos em 20 rodadas, e a fração 0,5 dobra isso (73 mil). Com 0,9 W, o LeanFed-c já os treina (119 mil) e a fração 0,5 acrescenta só 12 mil. Nos clientes de 1500 e 1000 J, as duas políticas treinam quase o mesmo (até 5 mil de diferença).
- **No pior caso, o ganho é menor que com o rádio igual** porque, com 2,7 W, nem a fração 0,5 consegue fazer os clientes de 500 J treinarem tanto (73 mil, contra 91 mil com 1,8 W): a folga total cai de +58 para +44 mil.
- **O LeanFed-c sozinho melhora com o rádio desigual** (75,5% e 76,4%, contra 74,8%); a fração 0,5 fica entre 76,9% e 77,3% nos três casos.
- Somando as potências iguais e desiguais com as mesmas baterias, o ganho segue a folga de exemplos: +12, +19, +44, +58 e +69 mil dão +0,4, +0,5, +1,6, +2,5 e +3,9 pontos. A fração mínima ajuda quando a rede é cara justamente para quem tem pouca bateria.

A fração mínima 0,5 nos outros α

α	FedAvg	LeanFed-c	Fração mínima 0,5	Fração 0,5 − LeanFed-c [IC 95%], seeds melhores	Fração 0,5 − FedAvg [IC 95%], seeds melhores
0,05	48,0 ± 10,8%	57,1 ± 10,9%	64,3 ± 8,3%	**+7,3** [+2,5; +12,1], 8 de 8	**+16,3** [+10,5; +22,2], 8 de 8
0,1	70,4 ± 6,0%	74,8 ± 3,6%	77,3 ± 2,7%	**+2,5** [+1,0; +4,1], 7 de 8	**+6,8** [+2,6; +11,1], 8 de 8
0,3	84,3 ± 1,5%	84,9 ± 0,8%	85,9 ± 0,6%	**+1,0** [+0,7; +1,4], 8 de 8	**+1,6** [+0,8; +2,5], 7 de 8
1,0	87,6 ± 0,5%	87,2 ± 0,5%	87,7 ± 0,5%	**+0,5** [+0,3; +0,7], 8 de 8	**+0,1** [−0,2; +0,5], 4 de 8

- **Melhora o LeanFed-c em todos os α**, em 8 de 8 seeds em α = 0,05, 0,3 e 1,0. A diferença diminui quando as classes se espalham, como o próprio ganho do LeanFed-c (seção 4).
- **Com α = 0,05, o ganho sobre o FedAvg quase dobra**: +16,3 pontos contra +9,1 do LeanFed-c sozinho. O pico também passa a ser maior que o do FedAvg (+3,7 pontos, 8 de 8); sem a fração mínima, o LeanFed-c ficava 4,2 pontos abaixo (0 de 8).
- **Com α = 0,3**, o ganho sobre o FedAvg passa a ser significativo: +1,6 pontos (7 de 8, p = 0,003), contra +0,6 sem significância do LeanFed-c.
- **Com α = 1,0, elimina o custo do LeanFed-c**: empata com o FedAvg (+0,1, 4 de 8; pico 88,0% nos dois), onde o LeanFed-c ficava 0,4 ponto abaixo.

A fração mínima 0,5 no FEMNIST

Partição	FedAvg	LeanFed-c	Fração mínima 0,5	Fração 0,5 − LeanFed-c [IC 95%], seeds melhores	Fração 0,5 − FedAvg [IC 95%], seeds melhores
Dirichlet α = 0,1	74,7 ± 2,5%	76,0 ± 1,8%	78,7 ± 1,6%	**+2,7** [+1,3; +4,1], 8 de 8	**+4,0** [+1,8; +6,2], 8 de 8
por escritor	84,5 ± 0,3%	84,3 ± 0,3%	84,7 ± 0,3%	**+0,4** [+0,3; +0,5], 8 de 8	**+0,2** [+0,1; +0,3], 7 de 8

- **Com Dirichlet α = 0,1, é a melhor política**: +2,7 pontos sobre o LeanFed-c e +4,0 sobre o FedAvg, nas 8 seeds nos dois casos (p = 0,003 e 0,004). O LeanFed-c sozinho ganha +1,3 do FedAvg, sem significância. O pico também sobe: 79,4% contra 77,6% do FedAvg e 76,7% do LeanFed-c.
- **Por escritor, os três empatam na prática** (84,3% a 84,7%). A fração 0,5 ainda fica à frente do FedAvg (+0,2; 7 de 8) e do LeanFed-c (+0,4; 8 de 8), e o LeanFed-c sozinho fica 0,2 ponto abaixo do FedAvg (2 de 8), como no CIFAR-10 com α = 1,0.
- É o que se esperava antes das runs: ganho na partição Dirichlet e empate por escritor, onde nenhuma classe fica concentrada nos clientes que o FedAvg perde (seção 4).

O que só foi medido no α = 0,1 do CIFAR-10. As frações 0,3, 0,7 e 1,0, a margem de cauda, o custo fixo D = 5 e os experimentos sem rádio, com margem de cauda na fração mínima e com outras potências do rádio, iguais ou diferentes entre clientes.

4. O ganho depende da concentração de classes

Pergunta. O LeanFed-c mantém os clientes vivos em todos os cenários. Quando isso vale acurácia? Foram testados dois tipos de cenário, cada um com FedAvg e LeanFed-c (8 seeds em cada):

- **FEMNIST** (caracteres escritos à mão, 62 classes, 12 pessoas por cliente), dividido **por escritor**, que é a partição natural, ou por **Dirichlet α = 0,1**, com as mesmas pessoas.
- **CIFAR-10 variando o α** da partição Dirichlet: 0,05, 0,1, 0,3 e 1,0. Só o α muda; quanto menor, mais cada classe fica concentrada em poucos clientes.

Cenário	n	FedAvg (%)	LeanFed-c (%)	Ganho [IC 95%]	Seeds com ganho	Dados perdidos pelo FedAvg	Classes com >80% perdidas
CIFAR-10, α = 0,05	8	48,0 ± 10,8	57,1 ± 10,9	**+9,1** [+4,6; +13,5]	8 de 8	58%	39%
CIFAR-10, α = 0,1	8	70,4 ± 6,0	74,8 ± 3,6	**+4,3** [−0,1; +8,8]	6 de 8	55%	32%
CIFAR-10, α = 0,3	8	84,3 ± 1,5	84,9 ± 0,8	**+0,6** [−0,3; +1,5]	6 de 8	45%	5%
CIFAR-10, α = 1,0	8	87,6 ± 0,5	87,2 ± 0,5	**−0,4** [−0,8; +0,0]	1 de 8	41%	0%
FEMNIST, Dirichlet α = 0,1	8	74,7 ± 2,5	76,0 ± 1,8	**+1,3** [−0,1; +2,7]	6 de 8	42%	16%
FEMNIST, por escritor	8	84,5 ± 0,3	84,3 ± 0,3	**−0,2** [−0,3; −0,0]	2 de 8	33%	0%

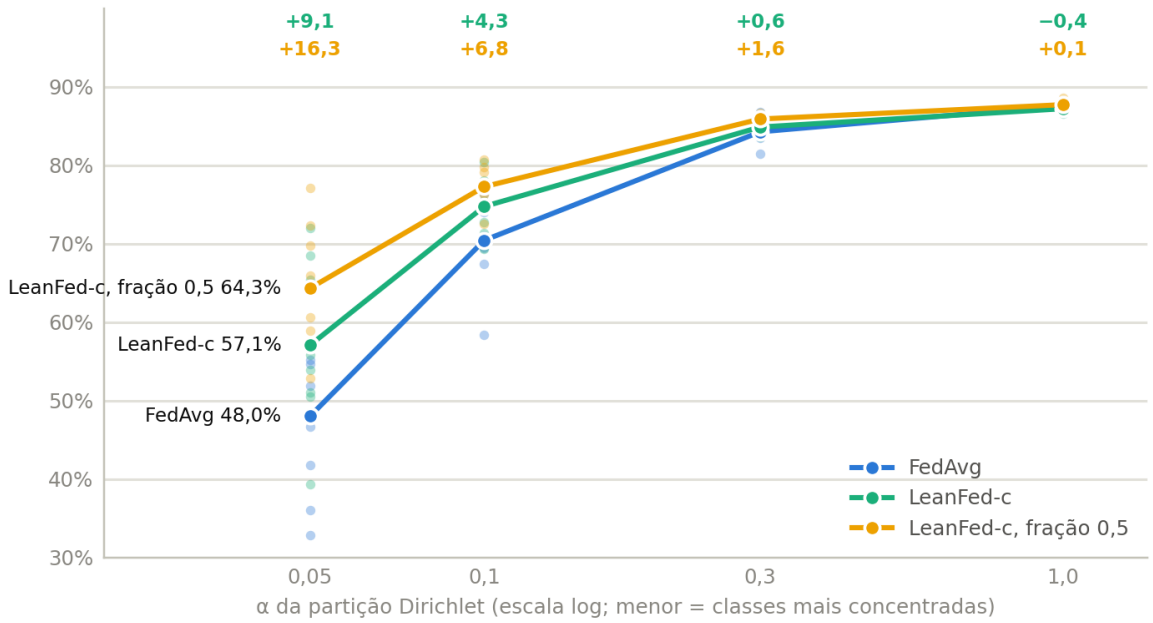


Figura 3. Acurácia média das 3 últimas rodadas contra o α do CIFAR-10, 8 seeds por ponto. Os pontos claros são as seeds; no alto, o ganho médio sobre o FedAvg, na cor de cada método. Todos pioram quando o α diminui; o FedAvg piora mais. A fração mínima 0,5 (seção 3) fica acima do LeanFed-c em todos os α.

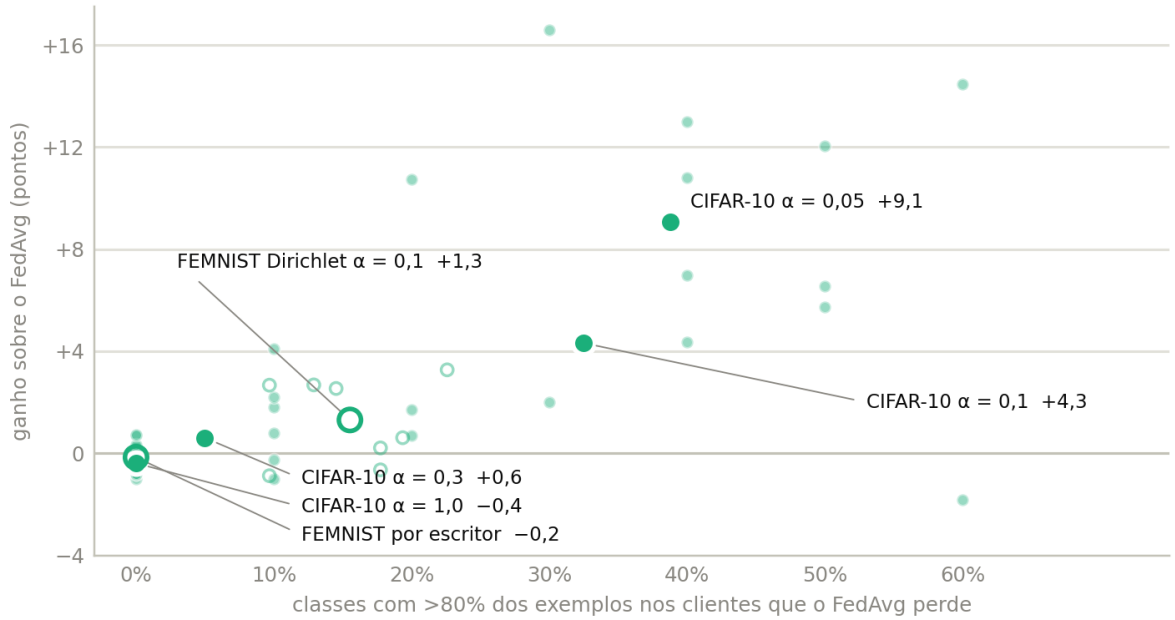


Figura 4. Ganho do LeanFed-c contra a fração de classes concentradas nos clientes que o FedAvg perde. Cada ponto pequeno é uma seed (8 por cenário); o grande, a média do cenário. CIFAR-10 preenchido, FEMNIST vazado.

O que os números dizem.

- **Perder dados não custa; perder classes custa.** O FedAvg perde de 32% a 58% dos dados de treino em todos os cenários, mas só fica atrás quando as classes estão concentradas nos clientes que saem. Com α = 0,3, ele perde 45% dos dados e 5% das classes, e o ganho do LeanFed-c é de 0,6 ponto, sem significância (IC 95% de −0,3 a

+1,5).

- **A acurácia por classe confirma.** No modelo da rodada 20 com α = 0,1, 74% do ganho do LeanFed-c sobre o FedAvg (87% do da fração mínima 0,5) está nas classes concentradas nos clientes que o FedAvg perde, que são 32% das classes. Nelas, o FedAvg acerta 54,8%, o LeanFed-c 67,2% e a fração 0,5 72,1%; nas outras, os três ficam entre 75% e 77%. Com α = 0,05, 93% do ganho do LeanFed-c está em 39% das classes.
- **O resultado se repete no mesmo dataset.** A comparação entre CIFAR-10 e FEMNIST mudava o dataset junto com a partição. A varredura do α muda só a concentração e reproduz a mesma relação: +9,1, +4,3, +0,6 e −0,4 pontos de α = 0,05 a α = 1,0.
- **O FEMNIST segue a mesma relação.** Com Dirichlet α = 0,1, 16% das classes ficam concentradas e o ganho do LeanFed-c é de +1,3 ponto (IC 95% de −0,1 a +2,7; 6 de 8), entre os do α = 0,1 e do α = 0,3 do CIFAR-10. Por escritor, nenhuma classe fica concentrada e o LeanFed-c fica 0,2 ponto abaixo, como no α = 1,0.
- **Só α = 0,05 tem ganho firme.** Positivo nas 8 seeds (de +0,7 a +16,6; t pareado p = 0,002). Em α = 0,1 o intervalo encosta no zero.
- **Sem concentração, o LeanFed-c custa pouco.** Com α = 1,0 ele fica abaixo do FedAvg em 7 das 8 seeds (−0,4 ponto, IC 95% de −0,8 a 0,0), e a acurácia máxima também é menor (87,4% contra 88,0%). Sem classes a proteger, sobra só o custo de treinar menos dados por rodada. Com a fração mínima 0,5, esse custo some (seção 3).
- **O ganho é relativo.** Com α menor os dois métodos pioram (Figura 3). O LeanFed-c não fica melhor com dados mais heterogêneos; ele resiste melhor a um cenário pior.
- **A relação vale entre cenários, não seed a seed.** Dentro de um mesmo α, a fração de classes concentradas não prevê o ganho (correlação de Spearman 0,30 com α = 0,05 e 0,14 com α = 0,1, ambas não significativas). Na seed 45 com α = 0,1, 60% das classes ficam concentradas e o ganho é negativo (seção 2); na seed 44 com α = 0,05, 50% e o ganho é dos menores (+6,5), porque os dois modelos vão mal (39% e 33%).

5. Reduzir o modelo (HeteroFL) perde para reduzir os dados

O LeanFed-c e a fração mínima cabem no orçamento **treinando menos dados**. A alternativa é **treinar um modelo menor**, como no HeteroFL (Diao et al., 2021): cada cliente treina todos os seus dados num submodelo de largura p, que é o prefixo de cada tensor do modelo completo (os primeiros canais de cada camada). Na MobileNetV3, esse submodelo é o *width_mult* = *p* do torchvision. Ele custa menos para treinar e para transmitir:

Largura p	Parâmetros	MACs	Energia de treino por amostra	Rede por rodada (média)
1	100%	100%	100% (8,88 mJ)	23,6 J
3/4	58,4%	68,2%	85,2%	13,8 J
1/2	26,8%	32,7%	68,2%	6,3 J
1/4	7,8%	15,1%	52,2%	1,8 J

A energia por amostra vem do mesmo perfil da GPU, medido para cada largura. A GPU aproveita mal os modelos pequenos: com 15% das operações, a largura 1/4 ainda gasta metade da energia de treino. A rede cai com os parâmetros, porque o cliente só paga o recorte (por simplificação, o servidor manda o modelo completo e o cliente corta).

Política. A cada rodada, o cliente escolhe a **maior largura** entre 1, 3/4, 1/2 e 1/4 cuja rodada completa (rede + treino de todos os dados) cabe no orçamento do LeanFed-c; se nenhuma cabe, pula a rodada. É a versão dinâmica do HeteroFL; a original fixa a largura de cada cliente pela capacidade dele. O servidor tira a média de cada elemento dos tensores entre os clientes cujo submodelo o contém, pesada pelos exemplos, e mantém o valor anterior onde ninguém treinou.

BatchNorm. Como no código do artigo, as camadas BatchNorm treinam com as estatísticas do lote, sem acumular médias (*static-bn*), e o modelo global é avaliado com elas recalibradas (seção 1). Antes de cada BatchNorm de um submodelo, a entrada é dividida por p (*Scaler*). Para comparar na mesma medida, nesta seção o LeanFed-c e a fração 0,5 também são avaliados com a BatchNorm recalibrada. Para eles, isso muda pouco: 74,7% e 78,0%, contra 74,8% e 77,3% na avaliação original.

Validação na seed 42

- **Só com p = 1**, a política é a fração mínima 1,0 (treina tudo ou pula), e a run reproduz a dela bit a bit, em todas as rodadas e clientes. Com o *static-bn* e p = 1, os pesos também são os mesmos: ele só muda a avaliação (82,5% com a BatchNorm recalibrada).
- **Sem os ajustes da BatchNorm, o modelo colapsa**: 22,2% nas 3 últimas rodadas, porque a média das estatísticas acumuladas mistura larguras diferentes. Recalibrando só o modelo final, ele sobe para 70,6%.
- **Com os ajustes**: 70,9% só com o *static-bn* e **72,2%** com o *static-bn* e o Scaler, a configuração do artigo, usada nas 8 seeds. Ainda 10 pontos abaixo da fração 1,0 com a mesma medida (82,5%).

Resultado com 8 seeds

Nas tabelas, *HeteroFL* usa as quatro larguras, e *HeteroFL, 1 e 3/4* só as duas maiores (subseção "Só as larguras 1 e 3/4").

α = 0,1, BatchNorm recalibrada	Média das 3 últimas	Acurácia máxima	Exemplos em 20 rodadas (no modelo completo)	Rede / treino (kJ)	Particip.	Sem bateria no fim
HeteroFL	67,7 ± 5,1%	70,8 ± 3,1%	780 mil (348)	5,24 / 5,89	297	0,8
HeteroFL, 1 e 3/4	72,7 ± 4,3%	75,1 ± 3,3%	676 mil (371)	5,47 / 5,62	271	0,6
LeanFed-c	74,7 ± 4,3%	77,4 ± 2,7%	505 mil (505)	6,71 / 4,51	285	1,6
fração mínima 0,5	78,0 ± 3,0%	80,6 ± 1,8%	563 mil (563)	6,15 / 5,04	260	1,2

Diferença pareada	Pontos [IC 95%], seeds em que o primeiro ganha
HeteroFL − LeanFed-c	**−6,9** [−10,7; −3,2], 0 de 8
HeteroFL − fração mínima 0,5	**−10,3** [−13,7; −6,8], 0 de 8
HeteroFL, 1 e 3/4 − HeteroFL	**+5,0** [+2,3; +7,6], 8 de 8
HeteroFL, 1 e 3/4 − LeanFed-c	**−2,0** [−4,2; +0,2], 2 de 8
HeteroFL, 1 e 3/4 − fração mínima 0,5	**−5,3** [−7,7; −2,9], 1 de 8

- **O HeteroFL perde nas 8 seeds**: −6,9 pontos para o LeanFed-c (t pareado, p = 0,003) e −10,3 para a fração 0,5 (p < 0,001). O pico também é menor (70,8% contra 77,4% e 80,6%). As perdas vão de −2,6 a −13,7 pontos para o LeanFed-c; as maiores são as das seeds 44 e 48 (−13,5 e −13,7).
- **Ele não perde por falta de energia nem de participação.** Gasta o mesmo (11,1 kJ, contra 11,2 kJ), participa mais (297 de 300 possíveis) e deixa menos clientes sem bateria (0,8). Como o submodelo transmite menos, a rede cai para 5,2 kJ e sobra mais energia para o treino (5,9 kJ, contra 4,5 e 5,0).
- **Treina mais exemplos, mas em modelos menores.** São 780 mil exemplos em 20 rodadas, contra 505 mil no LeanFed-c e 563 mil na fração 0,5, quase todos os dados em toda rodada. Só 348 mil passam pelo modelo completo: 59% das participações são com p = 1, 21% com 3/4, 13% com 1/2 e 6% com 1/4.
- **Os clientes de 500 J ficam quase sempre nos submodelos**: treinam 228 mil exemplos (38 mil no LeanFed-c, 91 mil na fração 0,5), mas 83% das participações deles são com p < 1 (33% com 3/4, 31% com 1/2, 19% com 1/4).
- **O ganho de "mais exemplos" da seção 3 não passa para os submodelos.** Contando só os exemplos no modelo completo, a ordem bate com a da acurácia (348, 505 e 563 mil para 67,7%, 74,7% e 78,0%).

A perda não fica nas classes dos clientes de bateria pequena

A hipótese era que, com Dirichlet α = 0,1, as classes dos clientes de 500 J chegam só aos canais do prefixo, e o resto do modelo completo aprende quase só com os clientes de mais bateria. A acurácia por classe do modelo final (rodada 20, BatchNorm recalibrada) a **rejeita**:

- Contra o LeanFed-c, o HeteroFL perde **3,8 pontos** nas classes com mais da metade dos exemplos nos clientes de 500 J, 3,7 nas dos clientes de 1000 J e **6,4** nas dos de 1500 J. Na média, −4,3 por classe, com 51 das 80 (10 classes × 8 seeds) abaixo do LeanFed-c.
- Nas seeds 44 e 48, as piores, a perda está nas outras classes (−10,7 e −10,1 pontos), e não nas dos clientes de 500 J (−0,6 e +1,4).
- A pior classe cai: 23,5% em média, contra 33,4% no LeanFed-c e 39,9% na fração 0,5.

A leitura é que misturar larguras na agregação atrapalha o modelo inteiro, e não só as classes de quem treina nos submodelos.

Só as larguras 1 e 3/4

Mesma política, com só p = 1 e p = 3/4: o cliente que não cabe nem em 3/4 pula a rodada, em vez de treinar em 1/2 ou 1/4.

- **Melhora 5,0 pontos** sobre as quatro larguras (72,7%, contra 67,7%), nas 8 seeds (p = 0,003). Os maiores ganhos são nas seeds 44 e 48 (+8,6 e +9,4), as piores com as quatro larguras.
- **Continua abaixo** do LeanFed-c (−2,0 pontos, ganha em 2 de 8 seeds, p = 0,07) e da fração 0,5 (−5,3, só ganha na seed 49, p = 0,001).
- Treina menos que as quatro larguras: 676 mil exemplos em 20 rodadas e 271 participações, porque quem não cabe em 3/4 pula. No modelo completo passam 371 mil, só 23 mil a mais. São 67% das participações com p = 1 e 33% com 3/4; nos clientes de 500 J, 74% com 3/4.
- Com quase os mesmos exemplos no modelo completo, tirar as larguras 1/2 e 1/4 vale 5 pontos: os submodelos pequenos atrapalham por si, como a acurácia por classe indicava. A ordem por exemplos no modelo completo continua a da acurácia (348, 371, 505 e 563 mil).

> **Diferenças para o HeteroFL do artigo.** A largura muda a cada rodada conforme a bateria, em vez de ser fixa por cliente. A máscara de rótulos do código do artigo no não-IID (a saída só considera as classes do cliente) não foi implementada, porque mudaria o treino das outras políticas também. E a economia de energia dos submodelos é a de uma GPU de desktop; num celular, treinar a largura 1/4 pode custar bem menos que metade.

6. Conclusões, limitações e próximos passos

O que já se pode afirmar

- Com comunicação realista, o LeanFed original não é melhor que o FedAvg. A correção que reserva comunicação e custo fixo é o que faz a política funcionar.

- O LeanFed-c traz ganho quando a heterogeneidade é de rótulo e as classes se concentram em clientes com bateria limitada. Com 8 seeds no CIFAR-10, o ganho é claro com α = 0,05 (+9,1 pontos, 8 de 8 seeds) e provável com α = 0,1 (+4,3, 6 de 8, p = 0,055). No FEMNIST com Dirichlet, +1,3 (6 de 8, p = 0,06). Sem essa concentração, ele empata com o FedAvg ou fica até 0,4 ponto abaixo.
- O ganho do LeanFed-c vem de manter os clientes de bateria pequena, e as classes deles, até o fim do treino: a acurácia por classe mostra que três quartos do ganho estão nas classes que o FedAvg perde. O pico de acurácia não melhora.
- A fração mínima 0,5 melhora o LeanFed-c nos quatro α do CIFAR-10 (+7,3 a +0,5 pontos) e o deixa à frente do FedAvg em α = 0,05, 0,1 e 0,3 (+16,3, +6,8 e +1,6), empatado em α = 1,0, com pico maior ou igual em todos. No FEMNIST, ganha +4,0 do FedAvg com Dirichlet (8 de 8) e +0,2 por escritor. É a melhor política medida em todos os cenários.
- O ganho da fração mínima vem de pular rodadas, e não de treinar uma fração dos dados: os clientes de bateria pequena guardam energia no começo e treinam todos os dados no fim. No α = 0,1, "treina tudo ou pula" (fração 1,0) é a melhor variante (+4,1 sobre o LeanFed-c, +8,5 sobre o FedAvg). O ganho sobre o LeanFed-c se espalha pelas classes, e a pior classe sobe mais que a média (+13,5 com a fração 1,0). Ele vem de treinar mais exemplos no total com a energia de rede que as rodadas puladas economizam: sem custo de rádio, o ganho cai para +1,2 e +0,6 ponto, sem significância; sem mortes (margem de cauda), ele fica; e ele cresce com a potência do rádio (+0,4, +2,5 e +3,9 pontos com 0,9, 1,8 e 2,7 W, na fração 0,5). Com o rádio desigual entre clientes, ele depende de quem paga a rede cara: +1,6 se são os de menos bateria, +0,5 se são os de mais.
- Reduzir o modelo em vez dos dados não compensa neste cenário: o HeteroFL dinâmico, com os ajustes da BatchNorm do artigo, fica 6,9 pontos abaixo do LeanFed-c e 10,3 abaixo da fração 0,5 no α = 0,1 (8 de 8 seeds), embora treine mais exemplos com a mesma energia. O que conta é quantos exemplos passam pelo modelo completo, e os submodelos pequenos atrapalham o modelo inteiro: sem as larguras 1/2 e 1/4, ele sobe 5,0 pontos, mas ainda fica 2,0 abaixo do LeanFed-c e 5,3 abaixo da fração 0,5.
- Para comparar políticas, usar a média das últimas rodadas e várias seeds; a acurácia de uma rodada e de uma seed é ruído. Três seeds não bastam: no α = 0,1, as seeds 42 a 44 deram o dobro do ganho do LeanFed-c medido com 8, e o ganho com D = 5 caiu de 10 para 6,3 pontos. Já a fração mínima, que com 3 seeds parecia ruído, se confirmou com 8.
- Com MobileNetV3 e dados não-IID, a média das estatísticas da BatchNorm feita pela FedAvg distorce a acurácia do modelo global no FEMNIST; para comparar métodos, é preciso recalibrá-las antes de avaliar (seção 1).

Limitações

- A fração mínima 0,5 tem os quatro α do CIFAR-10 e as duas partições do FEMNIST; as frações 0,3, 0,7 e 1,0, a margem de cauda e D = 5, só o α = 0,1 do CIFAR-10.
- No FEMNIST, a BatchNorm só é recalibrada na avaliação: os clientes continuam recebendo a média das estatísticas. Não se sabe se recalibrar também no modelo

enviado (ou trocar a BatchNorm por GroupNorm) mudaria o treino, nem por que a distorção varia de 5 a 16 pontos entre os métodos.

- A causa do ganho da fração mínima (mais exemplos no total) só foi testada no α = 0,1 do CIFAR-10, e depende do custo do rádio: com 0,9 W o ganho some. Com o rádio diferente entre clientes, só foram testadas as duas atribuições extremas, com média 1,8 W e a mesma faixa de rede. A acurácia por classe só foi medida no modelo da rodada 20, porque os das outras rodadas não são salvos, e não se sabe se o ganho depende de parar o treino na rodada 20.
- A fração de classes concentradas explica a diferença entre cenários, mas não entre seeds do mesmo cenário: falta uma medida que preveja quando o FedAvg realmente degrada no fim.
- Setup único: uma faixa de rede (25 Mbps), diferença de potência entre clientes só nos dois casos da seção 3, sem recarga. A energia vem do perfil de uma GPU de desktop, não de um celular.
- As mortes no fim por uma rodada de rede ruim continuam sem solução: a margem de cauda troca a morte por rodadas puladas.
- O HeteroFL só foi medido no α = 0,1 do CIFAR-10, só na versão dinâmica e sem a máscara de rótulos do artigo, com o perfil de energia da GPU, em que os submodelos economizam pouco treino (seção 5).

Próximos passos

- Entender os casos em que o FedAvg perde classes e não degrada (seed 45 com α = 0,1), com a acurácia por classe (*scripts/class_accuracy.py*).
- Reavaliar a ideia de fazer os clientes frágeis contribuírem mais cedo: a fração mínima ganha fazendo o contrário.
- Variar o setup (faixa de rede por cliente, baterias, recarga) para ver se a vantagem se mantém.
- Testar se o HeteroFL melhora com a largura fixa por cliente do artigo ou com um perfil de energia de celular, em que os submodelos economizam mais treino.
- Para decidir: um terceiro dataset como o EuroSAT só faz sentido se a dissertação mirar sensoriamento remoto, porque repete a condição do CIFAR-10 (10 classes, sem partição natural).

Apêndice A. Runs e como reproduzir

Pastas em *outputs/* (prefixo do id). Todas com o cenário da seção 1; cada linha muda só o que o nome diz. As runs do FEMNIST são as com *bn-recalibrate=true* (lote de 23/10), que repetem as anteriores com a mesma acurácia original e acrescentam a recalibrada. As da seção 5 também (lote de 25/10, código da branch *heterofl*); as do LeanFed-c e da fração 0,5 repetem as da seção 3 com a mesma acurácia original, bit a bit. As demais runs estão no apêndice do histórico.

Cenário e política	Seed 42	Seed 43	Seed 44
CIFAR-10 α = 0,05: FedAvg	17512...	15739...	68363...
CIFAR-10 α = 0,05: LeanFed-c	13324...	56224...	74911...
CIFAR-10 α = 0,05: fração mínima 0,5	74231...	39955...	21686...
CIFAR-10 α = 0,1: FedAvg	1977...	18006...	10759...
CIFAR-10 α = 0,1: LeanFed	16069...	11656...	11938...
CIFAR-10 α = 0,1: LeanFed-c	10494...	12096...	4501...
CIFAR-10 α = 0,1: fração mínima 0,3	70223...	12763...	12243...
CIFAR-10 α = 0,1: fração mínima 0,5	57608...	11724...	21629...
CIFAR-10 α = 0,1: fração mínima 0,7	94830...	11212...	36821...
CIFAR-10 α = 0,1: fração mínima 1,0	39105...	11630...	13244...
CIFAR-10 α = 0,1: margem de cauda 0,9	17135...	16908...	11427...
CIFAR-10 α = 0,1: fração 0,5 e margem de cauda 0,9	67067...	56917...	91968...
CIFAR-10 α = 0,1: fração 1,0 e margem de cauda 0,9	15522...	18252...	10244...
CIFAR-10 α = 0,1, sem rádio: LeanFed-c	56265...	11039...	11506...
CIFAR-10 α = 0,1, sem rádio: fração mínima 0,5	11059...	17811...	75048...
CIFAR-10 α = 0,1, sem rádio: fração mínima 1,0	53248...	34449...	97914...
CIFAR-10 α = 0,1, rádio 0,9 W: LeanFed-c	28894...	86803...	15504...
CIFAR-10 α = 0,1, rádio 0,9 W: fração mínima 0,5	52212...	12939...	23937...
CIFAR-10 α = 0,1, rádio 2,7 W: LeanFed-c	17904...	79092...	96421...
CIFAR-10 α = 0,1, rádio 2,7 W: fração mínima 0,5	70098...	43013...	52032...
CIFAR-10 α = 0,1, rádio pior caso: LeanFed-c	48003...	86228...	12501...
CIFAR-10 α = 0,1, rádio pior caso: fração mínima 0,5	93772...	82629...	76218...
CIFAR-10 α = 0,1, rádio favorável: LeanFed-c	10534...	15985...	95932...
CIFAR-10 α = 0,1, rádio favorável: fração mínima 0,5	71955...	10562...	16631...
CIFAR-10 α = 0,1, D = 5: FedAvg	99297...	14583...	15387...
CIFAR-10 α = 0,1, D = 5: LeanFed	41686...	16970...	12657...

Cenário e política	Seed 42	Seed 43	Seed 44
CIFAR-10 α = 0,1, D = 5: LeanFed-c	10067...	10184...	98610...
CIFAR-10 α = 0,3: FedAvg	11428...	40976...	15233...
CIFAR-10 α = 0,3: LeanFed-c	14875...	52897...	68942...
CIFAR-10 α = 0,3: fração mínima 0,5	10723...	10638...	15043...
CIFAR-10 α = 1,0: FedAvg	12018...	21996...	11654...
CIFAR-10 α = 1,0: LeanFed-c	17183...	69488...	15431...
CIFAR-10 α = 1,0: fração mínima 0,5	18343...	49975...	67601...
FEMNIST escritor: FedAvg	29553...	14431...	10766...
FEMNIST escritor: LeanFed-c	14878...	28328...	12001...
FEMNIST escritor: fração mínima 0,5	17757...	44950...	10449...
FEMNIST Dirichlet: FedAvg	12511...	15898...	72012...
FEMNIST Dirichlet: LeanFed-c	88227...	19479...	59824...
FEMNIST Dirichlet: fração mínima 0,5	45006...	95189...	12909...
Seção 5: HeteroFL	15138...	86454...	94847...
Seção 5: HeteroFL, 1 e 3/4	13116...	40801...	84638...
Seção 5: LeanFed-c	11947...	96813...	36729...
Seção 5: fração mínima 0,5	12734...	10894...	45556...

Cenário e política	Seed 45	Seed 46	Seed 47	Seed 48	Seed 49
CIFAR-10 α = 0,05: FedAvg	11973...	10618...	13286...	18104...	75581...
CIFAR-10 α = 0,05: LeanFed-c	17529...	19458...	80657...	10299...	40953...
CIFAR-10 α = 0,05: fração mínima 0,5	94595...	16242...	15379...	33992...	11574...
CIFAR-10 α = 0,1: FedAvg	84285...	72545...	11917...	11541...	13836...
CIFAR-10 α = 0,1: LeanFed	12960...	14051...	63580...	13382...	92632...
CIFAR-10 α = 0,1: LeanFed-c	12443...	15957...	22576...	91925...	40049...
CIFAR-10 α = 0,1: fração mínima 0,3	58980...	14473...	90559...	83106...	97801...
CIFAR-10 α = 0,1: fração mínima 0,5	17593...	38953...	74931...	13337...	13428...
CIFAR-10 α = 0,1: fração mínima 0,7	10531...	68311...	13025...	14309...	92826...
CIFAR-10 α = 0,1: fração mínima 1,0	79835...	14346...	13550...	17814...	57671...

Cenário e política	Seed 45	Seed 46	Seed 47	Seed 48	Seed 49
CIFAR-10 α = 0,1: margem de cauda 0,9	80290...	14219...	18360...	15014...	24191...
CIFAR-10 α = 0,1: fração 0,5 e margem de cauda 0,9	58638...	73913...	62442...	24803...	38960...
CIFAR-10 α = 0,1: fração 1,0 e margem de cauda 0,9	92410...	15962...	15879...	15497...	69158...
CIFAR-10 α = 0,1, sem rádio: LeanFed-c	36352...	76661...	82841...	62932...	99753...
CIFAR-10 α = 0,1, sem rádio: fração mínima 0,5	62209...	15923...	26993...	18326...	94229...
CIFAR-10 α = 0,1, sem rádio: fração mínima 1,0	25801...	15356...	12690...	18217...	17036...
CIFAR-10 α = 0,1, rádio 0,9 W: LeanFed-c	25716...	59442...	49788...	17171...	18359...
CIFAR-10 α = 0,1, rádio 0,9 W: fração mínima 0,5	29361...	35438...	24192...	16546...	12529...
CIFAR-10 α = 0,1, rádio 2,7 W: LeanFed-c	15844...	14213...	13321...	13060...	83579...
CIFAR-10 α = 0,1, rádio 2,7 W: fração mínima 0,5	10793...	37793...	35657...	87928...	12324...
CIFAR-10 α = 0,1, rádio pior caso: LeanFed-c	77058...	15818...	18227...	11363...	17824...
CIFAR-10 α = 0,1, rádio pior caso: fração mínima 0,5	12423...	98236...	93644...	15060...	78797...
CIFAR-10 α = 0,1, rádio favorável: LeanFed-c	46272...	17018...	86016...	17404...	98525...
CIFAR-10 α = 0,1, rádio favorável: fração mínima 0,5	93140...	15979...	39601...	11074...	11695...
CIFAR-10 α = 0,1, D = 5: FedAvg	16348...	58377...	55821...	18038...	63194...
CIFAR-10 α = 0,1, D = 5: LeanFed	43062...	39301...	17083...	13518...	13321...
CIFAR-10 α = 0,1, D = 5: LeanFed-c	55191...	12880...	10570...	10191...	16714...
CIFAR-10 α = 0,3: FedAvg	48566...	14967...	14939...	15273...	15319...
CIFAR-10 α = 0,3: LeanFed-c	48229...	10455...	13603...	14944...	10437...
CIFAR-10 α = 0,3: fração mínima 0,5	16108...	15101...	15890...	21442...	17025...
CIFAR-10 α = 1,0: FedAvg	60910...	13293...	12234...	84912...	14615...
CIFAR-10 α = 1,0: LeanFed-c	20015...	13867...	16467...	37089...	75707...

Cenário e política	Seed 45	Seed 46	Seed 47	Seed 48	Seed 49
CIFAR-10 α = 1,0: fração mínima 0,5	88964…	27903…	58342…	18979…	15558…
FEMNIST escritor: FedAvg	14499…	13120…	12118…	15009…	14904…
FEMNIST escritor: LeanFed-c	12487…	15266…	10577…	13214…	15799…
FEMNIST escritor: fração mínima 0,5	46534…	50927…	14157…	32677…	11099…
FEMNIST Dirichlet: FedAvg	16578…	18360…	16197…	44154…	81443…
FEMNIST Dirichlet: LeanFed-c	17665…	15770…	70768…	13113…	18339…
FEMNIST Dirichlet: fração mínima 0,5	19907…	77329…	16194…	15101…	59879…
Seção 5: HeteroFL	12242…	52334…	12191…	18365…	22798…
Seção 5: HeteroFL, 1 e 3/4	13392…	57163…	10030…	20290…	11432…
Seção 5: LeanFed-c	13599…	97546…	75654…	37169…	98855…
Seção 5: fração mínima 0,5	16110…	61626…	94603…	17376…	17441…

Validação do HeteroFL na seed 42 (seção 5): só p = 1, 12047…; sem os ajustes da BatchNorm, 37524…; só *static-bn*, 12564…; p = 1 com *static-bn*, 68622…

Com os containers de pé (*./run.sh -n 15 -b 1500,1000,500 --net-mbps 25*):

```
# lotes (um por cenário)
./scripts/run_seeds.sh -v fedavg,leanfed,leanfed-c 42 43 44           # CIFAR-10, α = 0,1
./scripts/run_seeds.sh -v fedavg,leanfed,leanfed-c 45 46 47 48 49     # seeds extras
./scripts/run_seeds.sh -v leanfed-c-mdf0.3,leanfed-c-mdf0.5,leanfed-c-tail0.9 42 43 44 45 46 47 48 49
./scripts/run_seeds.sh -x "energy-fixed-j=5.0" -v fedavg,leanfed,leanfed-c 42 43 44 45 46 47 48 49
./scripts/run_seeds.sh -v leanfed-c-mdf0.7,leanfed-c-mdf1.0 42 43 44 45 46 47 48 49
./scripts/run_seeds.sh -x "tail-margin=0.9" -v leanfed-c-mdf0.5,leanfed-c-mdf1.0 42 43 44 45 46 47 48 49
# sem rádio: subir antes ./run.sh -n 15 -b 1025,550,85 --net-mbps 25 --radio-w 0
./scripts/run_seeds.sh -v leanfed-c,leanfed-c-mdf0.5,leanfed-c-mdf1.0 42 43 44 45 46 47 48 49
# outra potência: subir antes ./run.sh -n 15 -b 1500,1000,500 --net-mbps 25 --radio-w 0.9 (ou 2.7)
./scripts/run_seeds.sh -v leanfed-c,leanfed-c-mdf0.5 42 43 44 45 46 47 48 49
# rádio por cliente: --radio-w 0.9,1.8,2.7 (pior caso) ou 2.7,1.8,0.9 (favorável), mesmas variantes
./scripts/run_seeds.sh -d cifar10-a0.3 -v fedavg,leanfed-c 42 43 44   # ou a0.05, a1.0
./scripts/run_seeds.sh -d cifar10-a0.3 -v leanfed-c-mdf0.5 42 43 44 45 46 47 48 49
./scripts/run_seeds.sh -d femnist -x "bn-recalibrate=true" \
    -v fedavg,leanfed-c,leanfed-c-mdf0.5 42 43 44 45 46 47 48 49   # ou -d femnist-dirichlet
# HeteroFL (branch heterofl; o perfil das larguras: profile_energy --widths 0.75,0.5,0.25)
./scripts/run_seeds.sh -x "bn-recalibrate=true" \
    -v heterofl,leanfed-c,leanfed-c-mdf0.5 42 43 44 45 46 47 48 49
./scripts/run_seeds.sh -x "widths='1.0,0.75'" -v heterofl 42 43 44 45 46 47 48 49   # 1 e 3/4
# média ± desvio, classes perdidas e ganho pareado (o --ref escolhe o cenário)
python3 scripts/seeds_summary.py --ref 1977  -o graficos/seeds         # α = 0,1 e variantes
python3 scripts/seeds_summary.py --ref 5626  -o graficos/seeds-sem-radio
python3 scripts/seeds_summary.py --ref 28894 -o graficos/seeds-radio-0.9    # 2,7 W: --ref 17904
python3 scripts/seeds_summary.py --ref 48003 -o graficos/seeds-radio-pior   # favorável: --ref 10534
python3 scripts/seeds_summary.py --ref 11428 -o graficos/seeds-alfa-0.3
python3 scripts/class_loss.py    --ref 11428 -o graficos/seeds-alfa-0.3
python3 scripts/seeds_summary.py --bn --ref 38162 -o graficos/seeds-femnist-dirichlet
python3 scripts/class_loss.py    --bn --ref 38162 -o graficos/seeds-femnist-dirichlet
python3 scripts/seeds_summary.py --bn --ref 10788 -o graficos/seeds-femnist   # escritor
python3 scripts/seeds_summary.py --bn --ref 1977 -o graficos/seeds-heterofl
# acurácia por classe do final_model.pt e parte de cada classe por bateria (na CPU)
docker run --rm -v aprendfed_data:/data:ro -v "$PWD":/app:ro aprendfed:latest \
    python scripts/class_accuracy.py [--bn] outputs/run-<id>-<data> ...
# Figura 4 a partir dos perda_classes.csv
python3 scripts/plot_gain_classes.py --ext pdf
```